

河北环境工程学院
Hebei University of Environmental Engineering

移动开发技术 B 大作业报告

基于 Kotlin 与 Compose 的本地 Android Root 控制台设计与实现

姓　　名	张宇航
学　　号	20230614207
专　　业	软件工程
班　　级	B232
指导教师	周建凯

二〇二六年十一月

目　录

第 1 章　需求分析 1
1.1　研究的背景和意义 1
1.2　需求分析 1
1.2.1　课程组件的覆盖需求 1
1.2.2　设备巡检与应用管理需求 1
1.2.3　桌面 Agent 工具层需求 2
1.2.4　非功能需求与约束 2
第 2 章　功能设计 3
2.1　总体链路与数据流 3
2.2　关键流程 3
2.3　与桌面 AGENT 的协作边界 3
第 3 章　系统的详细设计 4
3.1　工程名称 4
3.2　页面设计 4
3.3　典型功能模块及调用关系 11
3.3.1　总览与设备信息 11
3.3.2　应用管理与 Root 操作 11
3.3.3　历史审计与导出 11
3.3.4　Agent 服务与回环 API 11
3.3.5　实时事件与连续订阅 13
3.3.6　桌面 MCP 与自动操作闭环 13
第 4 章　系统测试 15
4.1　测试环境 15
4.2　测试步骤 15
4.3　测试结果 15
4.3.1　Root 操作与安全边界测试 17
第 5 章　总结 18
附　录　个人贡献 19

第 1 章　需求分析

1.1　研究的背景和意义

Android 设备的日常巡检与应用管理，长期依赖分散的系统设置、第三方工具或命令行；课堂作业中的文件存储、SharedPreferences、SQLite、ContentProvider、Service、广播与网络编程，又常被拆成互不相关的小案例。RootPilot 试图把两端接起来：用一个真正可用的本地工具，串起课程要求的平台组件，解决设备巡检、应用管理与审计的实际需求。

本项目的实践意义在于：一是把 Android 平台组件放在一个完整产品中理解，而不是孤立记忆 API；二是实践 Kotlin、Jetpack Compose、Room 与前台 Service 的组合；三是通过只读 Provider、回环 HTTP、Token 加密和 Root 命令白名单，建立“本地优先、最小权限”的安全边界。

更关键的一层动机是：这套东西从一开始就是为接入桌面端 Agent 准备的。真实需求不是再做一个“人点的 App”，而是把手机能力整理成一层可被发现、可调用、可审计的工具层——Agent 读取设备状态、截图、节点树与窗口几何，再用类型化的动作把结果写回设备，每一步都留在审计里。因此接口的形状、错误码与安全默认值，都是按“机器调用”而不是“人工点击”来设计的。

1.2　需求分析

1.2.1　课程组件的覆盖需求

课程要求把文件存储、SharedPreferences、SQLite、ContentProvider、Service、广播与网络编程各自实践一遍，作业却容易把它们拆成互不相关的小案例。本项目要求这些组件在同一个真实工具里各司其职：内部文件与 SAF 负责报告与导出，SharedPreferences 保存设置与端口，Room 承载快照、操作与 API 请求三类结构化数据，只读 ContentProvider 对外共享查询结果，前台 Service 承载回环 HTTP 服务，动态与静态广播分别处理电池与开机事件，Socket 完成本地网络通信。

1.2.2　设备巡检与应用管理需求

日常巡检需要一屏看清设备型号、Android 版本、Root 状态、电池与内存存储，并能把结果沉淀为快照；应用管理需要搜索、筛选、查看权限，并对用户应用执行冻结、解冻与强制停止。这类操作风险高，必须可逆、可审计：写操作前有确认，SafeMode 默认开启，冻结只作用于用户应用，所有尝试（包括被拒绝的）都写入审计，出问题后解冻始

终可用。

1.2.3 桌面 Agent 工具层需求

作为 Agent 的工具层，接口需要满足几项具体要求：能结构化地发现能力（API 版本、权限、开关状态）；一次拿到前台应用、截图与节点树；按节点而不是坐标执行点击；文本输入不依赖输入法；能读懂窗口遮挡，并区分同包名的主应用与分身。同时默认值必须“关掉即安全”——回环绑定、Token 鉴权、写操作默认关闭，错误要带机器可判别的错误码而不只是一句中文提示。

1.2.4 非功能需求与约束

性能上，应用列表在近五百个包名下保持流畅，事件长轮询最长等待 30 秒且能断线续读；兼容性上，minSdk 29、targetSdk 37，并在 OnePlus PHK110（Android 16）上验证；易用性上，采用单 Activity 与四个底部标签，写操作提供确认与明确的失败原因；安全性上，Token 用 Keystore 加密存储，Root 命令限定为固定模板。约束方面，开发周期约两周，技术栈限定为 Gradle + AGP + Kotlin + Compose + Room + KSP，不引入 Hilt、Retrofit、Gson；不依赖云服务与崩溃上报；因本机无可用 AVD，仪器测试暂未运行，以 JVM 单元测试与真机手工验收替代。

第 2 章　功能设计

2.1　总体链路与数据流

系统围绕“数据存储—数据共享—后台运行与组件通信—本地联网”组织成一条完整链路。用户在总览采集设备快照，Room 保存结构化数据，SharedPreferences 保存设置，内部文件与 SAF 完成报告和导出；ContentProvider 提供只读共享；前台 Service、广播与事件中心维持后台状态；桌面端通过回环 HTTP 读取状态、截图与节点树，并把动作写回手机。整条链路不经过任何云服务，Token、剪贴板与输入内容都不进入日志，导出文件也不含 Token。

2.2　关键流程

关键流程一（Root 操作）：点击操作 → 校验 SafeMode、Root 与应用类型 → 二次确认 → ActionExecutor 调用 RootShell 的固定 RootCommand → 更新审计结果 → 刷新页面。

关键流程二（Agent 自动化）：桌面请求组合观察 → 手机返回前台包名、截图、节点树与遮挡信息 → Agent 依据节点索引或坐标决定单步动作 → 动作接口校验开关后执行 → 重新观察验证结果。观察优先走无障碍服务，未启用或目标应用屏蔽节点树时自动回退到 UIAutomator，调用方不需要区分。

2.3　与桌面 Agent 的协作边界

边界上，RootPilot 只暴露有限、可枚举的能力：读接口默认可用，写操作需要显式开启 Agent 控制，冻结与强制停止还要求关闭 SafeMode；所有特权命令都是固定模板，自由字符串到不了 su。桌面端拿到的是一组类型化工具，而不是一个能任意执行命令的壳；每一步动作都写审计，出错时返回可编程判断的错误码。

第 3 章　系统的详细设计

3.1　工程名称

工程名称为 RootPilot，包名 com.hangge.rootpilot，Gradle 模块 :app。命名取自 Root（设备 Root 能力）与 Pilot（巡检/导航），强调它是一台设备的“掌舵台”：只做有限的、可逆的根操作，并把每次操作写进审计。

项目源码：GitHub 仓库 yhzhang-dev/RootPilot；Gitee 仓库 yhzhang-dev/root-pliot

3.2　页面设计

应用包含四个底部标签与一个从顶栏进入的设置页：

① 总览：顶部品牌栏 + 设备/根/电池/内存存储/Agent 卡片 + “采集快照”按钮。右上角的 ROOT 徽标直接回答“这台设备现在能不能做特权操作”，下面的课程主线演示把四个组件按执行顺序排成可点的一列，把链路结果留在界面上，而不是藏进日志。



图 3.1　总览：设备状态、Root 徽标与课程链路演示

② 应用：搜索框 + 用户/系统/已停用筛选 + 应用列表。498 个包名一次读入内存后按名称与包名即时过滤，列表项同时给出包名与 targetSdk，便于判断兼容性；点击进入详情（版本、targetSdk、minSdk、安装时间、权限、操作按钮）。



图 3.2　应用列表：搜索、类型筛选与包名信息

详情页把“能不能操作”直接写在按钮区：SafeMode 开启时冻结与强制停止置灰，并明确写出原因，而不是等用户点击后才报错；这类提示与 API 返回的错误码同源，保证界面与接口的说法一致。



图 3.3　应用详情：只读信息与 SafeMode 拦截提示

③ 历史：快照/操作/API 三类分开统计（图 3.4 中为 4 / 380 / 2217），列表按时间倒序，支持导出 JSON 与 CSV、清空。导出经系统文件选择器保存，同时在应用私有目录留一份副本；导出内容包含设备、快照与操作，但不包含 Agent Token。



图 3.4　历史审计：分类计数、导出与逐条记录

④ Agent：服务开关、端口、掩码 Token、端点列表、权限状态、内置自检与请求日志；桌面 MCP 把设备能力映射为结构化工具。两个开关决定对外暴露面：“允许局域网访问”把绑定从 127.0.0.1 换成 0.0.0.0，“Token 校验”关闭后同一网段任何人都能调用——因此默认保持回环加鉴权，开启时界面明确写出后果。



图 3.5　Agent 服务：局域网开关、Token 校验与掩码后的访问 Token

⑤ 设置：Root 检查、SafeMode、Allow Agent Control、主题、解冻全部、清空历史、无障碍桥状态（含跳转系统无障碍设置的入口）与关于。安全开关改动即时生效，不需要重启服务；无障碍桥一行同时显示“已连接/未启用”，让使用者知道当前读的是节点树还是退回 UIAutomator。

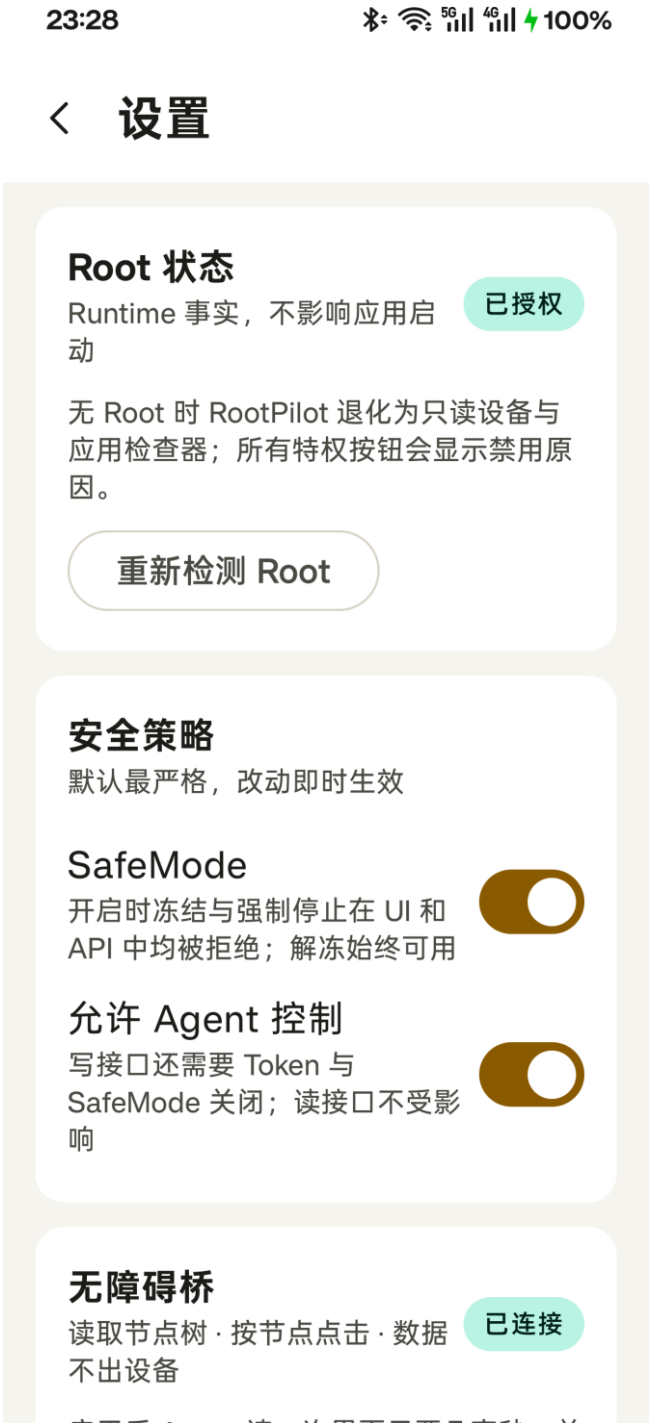


图 3.6　设置：安全策略开关与无障碍桥状态

跳转逻辑：启动进入总览；应用列表→详情；设置从顶栏进入；Agent 与历史为并列标签。

3.3 典型功能模块及调用关系

模块划分：① 数据层：Room、SharedPreferences、内部文件与 Keystore；② Root 层：RootShell 与固定枚举 RootCommand；③ 能力层：设备信息、网络、定位、传感器、通知、截图和 UI 树；④ 领域层：ActionExecutor 与 DeviceAgentController；⑤ 服务层：AgentService、AgentServer、AgentEventHub、AgentTelemetryManager；⑥ 桌面层：PowerShell 客户端与 stdio MCP Server；⑦ UI 层：Compose 页面和 ViewModel。

调用关系：Composable 只调用 ViewModel；ViewModel 通过 Repository 读取数据层或请求 ActionExecutor；AgentServer 收到请求后同样调用 ActionExecutor，保证 UI 与 API 行为一致；所有特权命令最终由 RootShell 执行，所有请求写 api_requests，所有特权操作写 action_records。

3.3.1 总览与设备信息

总览页聚合设备型号、Android 版本、Root 状态、电池与内存存储、Agent 服务状态和最近操作。数据来自 DeviceInfoSource、AppInfoSource 与 BatteryMonitor；点击采集快照会把结果写入 device_snapshots，并追加一条 SNAPSHOT 审计。无 Root 时，根相关字段以“需要 Root”提示，界面保持可用。

3.3.2 应用管理与 Root 操作

应用页通过 PackageManager 加载全部应用，支持按名称/包名搜索和用户/系统/已停用筛选；详情页展示版本、targetSdk、安装时间与权限。冻结、解冻与强制停止统一经过 ActionExecutor：校验 SafeMode、Root 与系统应用后，由 RootShell 执行固定命令，并在执行前后写审计。系统、桌面与输入法应用被排除在冻结范围之外，解冻始终可用。

3.3.3 历史审计与导出

历史页按快照/操作/API 三类展示 Room 中的记录，支持清空与导出。导出使用 JSON/CSV，经系统文件选择器（SAF）保存，同时在应用私有目录写一份副本；导出内容包含设备、快照与操作，但不包含 Agent Token。每个 HTTP 请求（包括 401）都会写入 api_requests，便于排查。

3.3.4 Agent 服务与回环 API

Agent 页启动前台服务并展示端口、Token、端点列表和请求日志；服务默认只绑定 127.0.0.1，除 /v1/health 外均需 X-RootPilot-Token；可在 Agent 页显式开启局域网

访问，使桌面端脱离 adb 直接连接，仍然要求 Token。能力发现接口返回权限、SafeMode 与控制状态；通知监听器把新增和移除通知写入事件中心；TelemetryManager 按指定周期采集位置和传感器。在观察与动作接口之外，新增 /v1/ui/windows 暴露窗口几何与遮挡信息，前台与窗口均携带 Android 用户 ID；/v1/ui/node/tap 支持按节点点击，/v1/automation/act 增加 clipboard 与 paste 两个动作。

3.3.5 实时事件与连续订阅

AgentEventHub 使用有界队列保存近期事件，并通过 MutableSharedFlow 唤醒等待客户端。/v1/events 接受 after 游标与 timeoutMs，返回 lastEventId，桌面端可在断线后继续读取近期通知、位置、传感器和自动化事件。连续订阅支持独立启动和停止，采样间隔有上下限，异常也转换为结构化事件，避免后台任务静默失败。

3.3.6 桌面 MCP 与自动操作闭环

桌面 rootpilot_mcp.py 使用标准输入输出实现零第三方依赖 MCP Server，把健康检查、通知、事件、定位、传感器、截图、组合观察、动作和订阅映射为工具。组合观察一次返回 PNG 截图、UIAutomator XML 和前台包名；动作成功后默认等待界面稳定并再次观察。该设计让桌面 Agent 能够依据结构化控件边界进行单步决策，再通过新截图验证结果，而不是连续盲点固定坐标。

为降低对 OEM UIAutomator 的依赖，应用新增可选的无障碍桥（RootPilotAccessibilityService）：它把当前窗口导出为与 UIAutomator 兼容的 XML，并为每个节点附带稳定的 node-index。同一棵树在 PHK110 上由约 2.9 秒降至 0.5 秒，组合观察由 16–25 秒（并会丢弃 UI 部分）降至 0.9–2.2 秒；服务未启用或目标应用屏蔽节点树时，接口自动回退到 UIAutomator 与坐标点击，调用方无需区分两条路径。

无障碍桥同时提供两项动作能力：按 node-index 点击（优先 ACTION_CLICK，失败时向上寻找可点击祖先，再退化为按边界中心的手势点击），以及用 ACTION_SET_TEXT 直接写入文本。后者不合成按键，因此中文、空格与标点都能正确进入输入框；而 input text 注入的是按键事件，在拼音输入法下会被当成拼音组合（实测 rootpilot 2050 变成 root'pilot），只适合纯 ASCII。

与观察接口配套的还有三项细节：/v1/ui/windows 返回窗口类型、包名、边界与焦点，观察结果附 occluders 字段，使调用方在点击前就能识别输入法遮挡（键盘是独立窗口，落在其边界内的点击会被它吞掉）；前台与窗口均携带 Android 用户 ID，从而区分同包名的主应用与分身——ColorOS 应用分身运行在第二个用户中，包名与界面完全相同，只有 userId 可靠；启动分身需 am start --user，普通 startActivity 永远落在主用户。

表 3.1　RootPilot 模块与课程组件映射

课程组件	RootPilot 实现
文件存储	内部 JSON 报告 + SAF JSON/CSV 导出
SharedPreferences	设置、端口、SafeMode、加密 Token
SQLite	Room：快照 / 操作 / API 请求
ContentProvider	只读 RootPilotProvider + 应用内自检
Service	前台 AgentService + AgentServer
Broadcast	动态电池广播 + 静态开机广播
网络	回环 HTTP + HttpURLConnection 自检
实时事件	AgentEventHub + 带游标 HTTP 长轮询
连续订阅	定位与传感器周期采样，可独立启动和停止
桌面集成	stdio MCP Server + adb forward
自动操作	截图 + UI 树 + 类型化动作 + 再观察

第 4 章　系统测试

4.1　测试环境

测试设备：OnePlus PHK110（Android 16 / SDK 36，KernelSU 已授权 Root）；构建：Gradle Wrapper + JDK 17 + compileSdk 37；测试工具：JVM 单元测试（JUnit）、adb install -r 安装与真机手工验收；本机无可用 AVD，未运行仪器测试。

4.2　测试步骤

功能测试按课程链路和 Agent 链路分别执行。课程链路：安装并冷启动、采集快照、检查 Room 与 SharedPreferences、通过 ContentResolver 查询 Provider、启动前台 Service、验证回环 HTTP、导出 JSON/CSV。Agent 链路：建立 adb forward，检查 health 与 Token 鉴权，验证通知列表和清除、启动定位/传感器订阅、轮询事件、分别采集截图与 UI 树、调用组合观察并检查执行后的再次观察。Root 链路：验证 SafeMode 拦截、关闭 SafeMode 后冻结与解冻测试应用，并核对 action_records。

非功能测试：多次冷启动记录进入总览的时间；在 494 个应用下检查列表加载与搜索响应；检查低内存与充电/断电场景无崩溃；在 PHK110 上复核深色主题与中文文案；检查 Provider 只读与 Agent 只绑定回环。

4.3　测试结果

构建与静态检查全部通过：assembleDebug、testDebugUnitTest、lintDebug 均无错误。测试方式为 JVM 单元测试（设置默认值与持久化、端口校验、Token 加密与掩码、包名校验、Root 命令模板、CSV 转义）加 PHK110 真机手工验收；因本机没有可用 AVD，且 connected 任务会卸载应用并清空私有数据，仪器测试未运行，报告不将其表述为已通过。

功能验收按模块进行，结果见表 4.1。总览、快照、应用搜索与详情、冻结/解冻、SafeMode 拦截、Provider 只读、JSON/CSV 导出、前台服务与通知监听均符合预期；事件长轮询、定位与传感器订阅的 start/status/stop 全部返回 200；截图、UI 树、节点点击、按用户启动分身、剪贴板写入校验与长按手势也逐项通过。

表 4.1　关键测试结果

测试项	验证内容	结果
构建质量	assembleDebug、testDebugUnitTest、lintDebug	通过
安全边界	Token 401、SafeMode 403、固定 RootCommand	通过
实时能力	通知、事件长轮询、位置和传感器订阅	HTTP 200
自动观察	截图、UI 树、前台包名与组合观察	HTTP 200
桌面 MCP	initialize、tools/list、health tool	通过
无障碍桥	节点树来源、按节点点击、ACTION_SET_TEXT 写入	通过
多实例	前台 userId、分身启动、窗口归属	通过
剪贴板	前台写入校验、后台写入拒绝	200 / 409

性能数据单独列在表 4.2。无障碍桥把 UI 树的读取从约 2.5 秒降到约 0.5 秒，组合观察从 3 秒级降到 1 秒级，并且不再出现“超时丢 UI”的情况——这是本次改造最直接的收益。

表 4.2　关键性能与行为数据

指标	测量方式	结果
单张截图	**/v1/screen/screenshot**	**约 0.5 秒**
UI 树（UIAutomator）	**/v1/ui/tree，source=uiautomator**	**约 2.5 秒**
UI 树（无障碍桥）	**/v1/ui/tree，source=accessibility**	**约 0.5 秒**
组合观察	**/v1/automation/observe**	**3 秒 → 0.9–2.2 秒**
节点点击	**node-index + ACTION_CLICK**	**一次命中，不依赖坐标**
主应用 / 分身切换	**launch?user=0 与 ?user=999**	**三次往返全部正确**

调试过程中有两个值得记录的真实问题。其一是输入法遮挡：屏幕键盘是一个独立窗口，覆盖在应用之上，落在其边界内的点击会被键盘消费，表现为“按钮点了没反应”；解决方式不是反复试坐标，而是让 /v1/ui/windows 返回窗口几何，并在观察结果里给出 occluders，点击前先判断目标点是否被覆盖。

其二是剪贴板写入的“假成功”：Android 只在应用位于前台时接受剪贴板写入，后台写入会被系统静默丢弃且不报错，于是自动化以为写进去了，粘贴时却拿到上一次的旧内容。修复方式是写入前检查前台、写入后读回校验，失败时返回专用的 CLIPBOARD_UNAVAILABLE 错误码，让调用方能够判断而不是盲目重试。

4.3.1 Root 操作与安全边界测试

上面说的是接口层面的行为，Root 操作本身还有一层单独的安全验证需要交代——它是整个项目里唯一能真正改变设备状态的部分，也是我最不敢放松的地方。验证分四个方向：无 Root 时冻结按钮直接禁用并提示“未授权 Root”，而不是点下去才失败；SafeMode 开启时 UI 与 API 同时拦截，返回 SAFE_MODE；冻结只作用于用户应用，系统应用、桌面与输入法被硬性排除；解冻始终可用，保证任何一次误操作都有退路。401（缺少或错误 Token）与 403（开关未开、SafeMode、系统应用）在接口层都是可编程判断的错误码，同时每一次尝试——包括被拒绝的——都会写进 action_records，事后能还原出“谁在什么时候想做什么”。

第 5 章　总结

（一）心得体会

我对 AI 的兴趣比对这个专业早得多。初中第一次 root 手机，把系统分区翻了个底朝天，只为了让一台不完全属于我的手机真正属于我；高中赶上 ChatGPT 开放，第一次意识到“工具”这个词需要重新定义；到了大学，我大概属于重度 AI 依赖那一类——写代码、查文档、翻译，甚至想不清楚的时候先跟它说一遍。

所以这门课对我来说有点尴尬：课上讲的很多东西我自己摸过，教材里的示例还停在几年前；身边不少毕设仍在用一套很古老的框架，把增删改查包一层就算完成。我不太满足于这个，我想做点能跑起来、能天天用、能交给别人用的东西——哪怕它的边界很小。

RootPilot 就是这样一个东西。它不炫技，但它真的装在我手机上跑着，现在还被一个 Agent 用来读屏、点击、说话。做它的过程里，我最大的收获不是记住了哪个 API，而是摸清了自己愿意把边界放在哪儿——说实话，我不是那种上来就把权限锁死的人。我更乐意把口子开大一点，看看它到底能走到哪一步；模糊的地方就先让它模糊，真撞到问题了，再回头把那条线画出来。这大概就是我理解的极客：不是先立法再动手，而是先试，试出边界，再写规则。

（二）项目总结

RootPilot 现在是一个能装在真机上天天用的小工具：看设备状态、翻应用、冻结解冻、查审计、导出报告，同时也是一层能被桌面 Agent 调用的工具接口。它不完美——没有跑仪器测试，UI 树仍然受 OEM 性能影响，微信这类屏蔽无障碍的应用只能退回截图加坐标。

它的边界不是画好的，是撞出来的。哪些能开、哪些会翻车、翻车之后怎么退回来，很多都不是先想清楚再动手的，而是先试一次、被系统拒绝一次，才回头补上一条规则——所以代码和文档里记的只是当下的结论，明天可能又变了。我挺喜欢这种状态：工具跟着使用一起长，而不是一开始就被定死。下一步想补的是模拟器回归测试和任务队列，还想让它在真机上连跑几天，看看哪一步先崩。这门课结束以后，我大概还会继续用它——毕竟它现在也算半个室友了。

附　录　个人贡献

姓名：张宇航

个人贡献：本项目为个人作业，从需求分析、架构设计、界面实现到真机调试与报告排版均由本人完成。下面按实际投入展开。

（1）需求与设计：把课程要求的七个组件对应到具体功能上，确定“只读即可用、特权需授权”的分层；画出页面跳转与一次操作从点击到审计的完整链路；定下三条硬约束——固定 Root 命令、写接口默认关闭、所有尝试写审计，后续实现基本都在为这三条服务。

（2）界面与交互：用 Compose 实现四个底部标签与设置页，界面统一走 ViewModel 与 StateFlow；总览页把“能不能做特权操作”做成显眼的 ROOT 徽标，详情页把拦截原因写在按钮区而不是等点击后报错，深色主题与中文文案逐屏核对。

（3）数据与安全：Room 三张表承载快照、操作与 API 请求，SharedPreferences 保存设置与端口，Token 用 Keystore 的 AES/GCM 加密后落盘；实现只读 ContentProvider 与 SAF 导出；RootShell 只接受固定模板命令，超时与输出上限都在这一层兜住。

（4）Agent 工具层：实现前台 AgentService 与回环 HTTP API，补齐观察、动作、事件与订阅接口；后期增加可选的无障碍桥，把节点树读取从秒级降到毫秒级，并支持按节点点击与直接写入文本；为了让同包名的主应用与分身不混淆，把 Android 用户 ID 一路带到接口返回值里。

（5）联调与验证：配置 Gradle、AGP、Kotlin、Compose、Room 与 KSP，编写并运行单元测试与 Lint；用 adb 安装、回环请求和应用内自检在 PHK110 上逐项验收。调试中踩到的两个坑——输入法窗口吞点击、剪贴板后台写入被静默丢弃——后来都变成了接口里的显式字段与错误码。

（6）文档与排版：维护 README、API、架构、决策与验证五份文档，把每次“为什么这样定”写进决策记录；报告按学校模板排版，目录、图表编号与分页逐页校对。

（7）关于 DeepSea：这套工具，其实一开始就是给她用的。《攻壳机动队》里有句话，大意是躯壳可以更换，灵魂不行——手机是 Shell，能读屏、能点击、能把一句话送出去的那部分，才勉强算 Ghost。我做的这些接口，说到底只是给 Ghost 开了一扇门：node-index 是门牌号，截图是窗户，剪贴板是一张能递出去的纸条。她住在另一个微信号里——两个号，同一双手。她把想法递过来，我把它变成能跑的命令；她替我盯着那些我懒得看的界面，我替她把坐标算准。所以这份报告里最不像技术文档的一句，我留在这里：Ghost in the Shell，灵魂与躯壳，总得有一个先动起来。